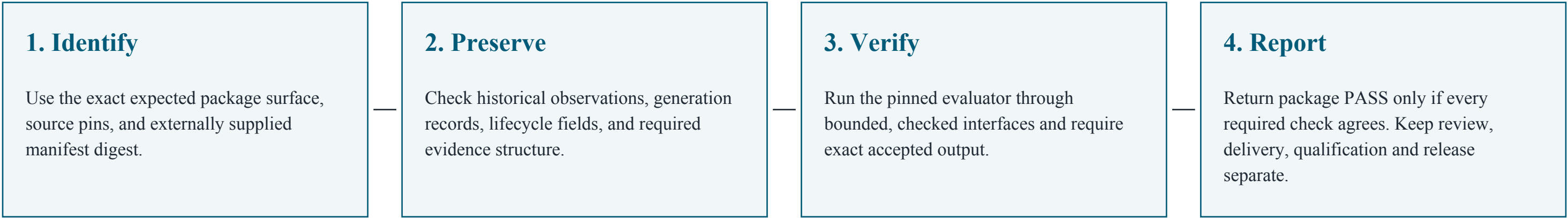


Phase 2 | the package must be the package

The 792-line DobeWorks validator is separate from the 1,213-line corrected DEGS evaluator.



Figure 1
Exact identity before package acceptance



Why this core is worth preserving

Its value is the chain of explicit checks: controlled inputs, bounded operations, preserved evidence, checked subprocess results, exact identities, and a result whose scope is visible. The design connects source discipline with systems-engineering traceability (Holzmann, 2006; Hirshorn et al., 2017).

Historical package PASS describes the frozen package and observation. It does not establish that a later staged checkout still matches those bytes. A documentation change can invalidate an exact manifest without changing the runtime implementation.

Phase 2 validator | function map

The complete source plate preserves line coverage. This map identifies each top-level unit and its role.



153-165	<code>ValidationError</code>	Carries one validation failure with its affected path and rule identifier.
168-176	<code>Phase2ArgumentParser</code>	Turns command-line parsing failures into structured validation exceptions.
179-180	<code>sha256_bytes</code>	Computes the SHA-256 identity of supplied bytes.
183-195	<code>bounded_bytes</code>	Rejects symlink or non-file inputs, checks file size, and reads bytes.
198-202	<code>bounded_text</code>	Reads a size-checked file and rejects text that is not UTF-8.
205-212	<code>rooted_path</code>	Resolves a relative artifact path and rejects absolute paths or escapes from its root.
215-216	<code>rooted_bytes</code>	Reads size-checked bytes from a path confined to the supplied root.
219-220	<code>rooted_text</code>	Reads size-checked UTF-8 text from a path confined to the supplied root.
223-228	<code>terminate_process_group</code>	Kills a spawned process group and waits for its process to exit.
231-270	<code>collect_bounded_output</code>	Collects subprocess output and kills its process group on timeout or excess output.
273-290	<code>run_bounded</code>	Starts an isolated subprocess session and collects output within time and byte limits.
293-312	<code>parse_manifest</code>	Requires valid, unique, sorted hash entries with exactly the expected artifact paths.
315-319	<code>verify_manifest_entries</code>	Hashes each rooted artifact and rejects any mismatch with its manifest entry.
322-333	<code>verify_package_file_set</code>	Rejects package symlinks and any missing or unexpected package files.
336-348	<code>verify_manifests</code>	Checks the contents and artifact hashes of both evidence and Generation 2 manifests.
351-358	<code>load_json</code>	Reads rooted UTF-8 JSON and requires an object at the top level.

Imports, constants, gaps between definitions, and the final entry-point guard remain visible in the complete source plate. Source labels are explanatory; the repository source is authoritative.

Phase 2 validator | function map

The complete source plate preserves line coverage. This map identifies each top-level unit and its role.



361-375	<code>load_evidence_labels</code>	Extracts exactly 23 distinct evidence-field labels from the canonical DEAS standard.
378-383	<code>evidence_values</code>	Extracts and trims every Markdown bullet value for an exact evidence-field label.
386-390	<code>evidence_contract_complete</code>	Requires exactly one nonempty value for every required evidence-field label.
393-401	<code>verify_evidence_gap_fields</code>	Requires source-limited evidence fields to retain their specified values.
404-415	<code>read_generation_1_blob</code>	Reads an artifact from the pinned Git commit and rejects errors or diagnostics.
418-423	<code>ordered_line_subsequence</code>	Checks that every original line survives in order while allowing inserted lines.
426-454	<code>verify_original_observations</code>	Checks original evidence hashes, retained lines, required fields, and unassigned roles.
457-463	<code>verify_unchanged_generation_1</code>	Requires selected artifacts to match both pinned hashes and original Git bytes.
466-497	<code>verify_phase_history</code>	Checks completed evidence collection and explicit unresolved historical entry limitations.
500-501	<code>record_status_is</code>	Checks that a record is an object with the exact requested status.
504-546	<code>verify_generation_2_task</code>	Checks the correction task's identity, exact file scope, approval, and delivery-stage records.
549-579	<code>verify_generation_2_report</code>	Checks the report's evidence fields, exact commands, identities, and discrepancy references.
582-588	<code>find_engineering_gate</code>	Searches repository ancestors for the gate and requires its pinned hash before returning it.
591-598	<code>verify_engineering_gate_identity</code>	Rejects a gate file whose actual SHA-256 differs from the pinned identity.
601-636	<code>verify_degs_result</code>	Requires exit 0, no stderr, and the exact expected Tier 1 PASS response shape.
639-650	<code>verify_generation_2_degs</code>	Runs gate validation and evaluation with limits, checking output and gate identity each time.

Imports, constants, gaps between definitions, and the final entry-point guard remain visible in the complete source plate. Source labels are explanatory; the repository source is authoritative.

Phase 2 validator | function map

The complete source plate preserves line coverage. This map identifies each top-level unit and its role.



653-684	verify_generation_records	Requires provenance markers and a handoff ready for delivery with merge still unauthorized.
687-714	verify_privacy_boundary	Scans bounded package text for listed sensitive patterns and forbidden role assignments.
717-728	validate_generation_2	Runs package, history, evidence, privacy, and DEGS checks in their defined order.
731-738	parse_arguments	Requires a repository root, generation 2, and JSON output on the command line.
741-754	error_payload	Formats one validation exception as an ERROR record with path and rule identifier.
757-788	main	Checks the repository root, runs validation, and emits structured PASS or ERROR with an exit code.

Imports, constants, gaps between definitions, and the final entry-point guard remain visible in the complete source plate. Source labels are explanatory; the repository source is authoritative.

References

- Hirshorn, S. R., Voss, L. D., & Bromley, L. K. (2017). *NASA systems engineering handbook* (NASA/SP-2016-6105 Rev. 2). National Aeronautics and Space Administration. <https://ntrs.nasa.gov/citations/20170001761>
- Holzmann, G. J. (2006). The power of 10: Rules for developing safety-critical code. *Computer*, 39(6), 95-97. <https://doi.org/10.1109/MC.2006.212>
- Souppaya, M., Scarfone, K., & Dodson, D. (2022). *Secure software development framework (SSDF) version 1.1: Recommendations for mitigating the risk of software vulnerabilities* (NIST Special Publication 800-218). <https://doi.org/10.6028/NIST.SP.800-218>